

Tarea 2: Análisis de Algoritmos

Mariel Solange Muñoz Castillo, Martín González Cifuentes
Javier Torres Ortiz, Ignacio Soto Miranda

Junio 2026

1. Diseño

1.1. Algoritmo Base

Con este algoritmo se resuelve el problema de la Distancia mínima entre todo par de nodos utilizando el algoritmo de Bellman-Ford.

- **Estructura y Lógica:** La función `alg_base` inicializa una matriz de distancias `dist` de tamaño $n \times n$ con valores infinitos. Luego, se itera sobre cada nodo del grafo tomándolo como un nodo origen s . Para cada origen, inicializa su distancia a sí mismo en 0 y se van relajando todas las aristas del grafo iterativamente $n - 1$ veces. La relajación consiste en verificar si el camino hacia un nodo v pasando por u es más corto que la distancia actual conocida hacia v .
- **Detección de Ciclos Negativos:** Tras finalizar las $n - 1$ iteraciones para un nodo origen s , el algoritmo realiza una pasada adicional sobre todas las aristas. Si detecta que alguna arista aún puede ser relajada, levanta un `ValueError` para detener la ejecución y evitar bucles infinitos en los experimentos.
- **Demostración** Sea $G = (V, E)$ un grafo dirigido y ponderado con $|V| = n$. Sean $u, v \in V$ dos nodos cualesquiera tales que existe al menos un camino desde u hasta v . El algoritmo de Bellman-Ford se basa en la relajación sucesiva de las aristas. Tras la i -ésima iteración, se garantiza que se han encontrado las distancias mínimas desde el nodo origen hacia todos los nodos cuyo camino óptimo utiliza a lo más i aristas.

Procedemos por inducción sobre el número de iteraciones. En la iteración 0, la distancia al nodo origen es 0, mientras que el resto de las distancias se consideran infinitas, por lo que la propiedad se cumple. Supongamos ahora que, después de i iteraciones, se han calculado correctamente las distancias mínimas para todos los caminos de longitud a lo más i aristas. Durante la iteración $i+1$, la relajación de cada arista (x, y) permite extender dichos caminos en una arista adicional, obteniendo así las distancias mínimas para todos los caminos de longitud a lo más $i+1$ aristas. Por el principio de inducción, la propiedad se mantiene para toda iteración.

Como un camino simple en un grafo con n nodos puede contener, a lo más, $n - 1$ aristas, tras ejecutar $n - 1$ iteraciones se obtienen las distancias mínimas entre el nodo origen y cualquier otro nodo alcanzable.

Para detectar ciclos de peso negativo, basta con realizar una iteración adicional de relajación sobre todas las aristas. Pues si alguna distancia aún puede reducirse, ello implica la existencia de un camino de longitud superior a $n - 1$ aristas con menor costo, lo que únicamente es posible si el recorrido incluye un ciclo de peso negativo. En tal caso, no existe una distancia mínima finita, ya que el costo del camino puede disminuir indefinidamente al recorrer dicho ciclo.

- **Complejidad Asintótica:** Dado que Bellman-Ford toma un tiempo de $O(|V| \cdot |E|)$ para un solo nodo origen, y esta implementación lo ejecuta n veces (una por cada nodo en V), la complejidad temporal total del Algoritmo Base es de $O(|V|^2 \cdot |E|)$.

1.2. Algoritmo de Floyd-Warshall

Este algoritmo aborda el mismo problema pero desde la perspectiva de la Programación Dinámica, basándose en la subestructura óptima de los caminos más cortos.

- **Estructura y Lógica:** La función `alg_floyd_warshall` construye una matriz de distancias directas. Primero, establece la diagonal principal en 0 (distancia de un nodo a sí mismo) y luego procesa la lista de aristas inicializando las distancias adyacentes. Un detalle robusto de la implementación es que, en caso de existir aristas paralelas entre dos nodos, conservando estrictamente la de menor peso.
- **Solución con programación dinámica:** Utiliza un triple ciclo iterativo (k, i, j) . En cada iteración, evalúa si el nodo k puede actuar como un nodo intermediario que ofrezca un camino más corto entre el nodo de origen i y el nodo de destino j . La actualización se realiza solo si los caminos parciales no son infinitos, garantizando que el peso total disminuya.
- **Detección de Ciclos Negativos:** A diferencia del algoritmo base, Floyd-Warshall detecta ciclos de peso negativo revisando la diagonal principal de la matriz resultante. Si la distancia de un nodo a sí mismo (`dist[i][i]`) resulta ser menor a 0, se confirma la existencia de un ciclo negativo y el script levanta de manera segura un `ValueError`.
- **Complejidad Asintótica:** Debido a los tres bucles anidados que iteran n veces cada uno (donde $n = |V|$), la complejidad de este algoritmo está estrictamente acotada por $O(|V|^3)$, independientemente de la densidad de las aristas ($|E|$). Además, requiere $O(|V|^2)$ de complejidad espacial para almacenar la matriz de distancias.

2. Implementación y Experimentos

Especificaciones del Sistema

- **Procesador:** Intel Core i5-12450HX (12.^a generación, 4 núcleos y 8 hilos disponibles).
- **Memoria RAM:** 9,7 GiB disponibles para el entorno de ejecución.
- **Sistema operativo:** Ubuntu 24.04 LTS sobre WSL2 (Windows, kernel 6.6.87.2-microsoft-standard-WSL2).
- **Compilador:** Python 3.13.13

2.1. Implementación y Correctitud

Para evaluar la implementación de ambos algoritmos se utilizó el archivo `prueba.py`, el cual ejecuta una serie de casos de prueba diseñados para validar la correctitud de los resultados obtenidos. Se consideraron las siguientes instancias de grafos dirigidos y ponderados:

1. **Grafo con pesos negativos (sin ciclos negativos).** Este caso verifica que ambos algoritmos sean capaces de calcular correctamente los caminos mínimos en presencia de aristas con peso negativo.

- $0 \rightarrow 1$ (peso 3)
- $0 \rightarrow 2$ (peso 8)
- $1 \rightarrow 3$ (peso 1)
- $2 \rightarrow 1$ (peso -2)
- $3 \rightarrow 2$ (peso 2)

2. **Grafo de cinco nodos con solución conocida.** Se empleó un grafo cuya matriz de distancias mínimas fue calculada manualmente, permitiendo comparar directamente la salida de ambos algoritmos con el resultado esperado.

- $0 \rightarrow 1$ (peso 2)
- $1 \rightarrow 2$ (peso 3)
- $2 \rightarrow 0$ (peso 4)
- $0 \rightarrow 3$ (peso 7)
- $3 \rightarrow 4$ (peso 1)
- $2 \rightarrow 4$ (peso 5)

La matriz de distancias esperada es:

$$\begin{bmatrix} 0 & 2 & 5 & 7 & 8 \\ 7 & 0 & 3 & 14 & 8 \\ 4 & 6 & 0 & 11 & 5 \\ \infty & \infty & \infty & 0 & 1 \\ \infty & \infty & \infty & \infty & 0 \end{bmatrix}$$

3. **Grafo donde el camino directo no es el óptimo.** Este caso comprueba que los algoritmos sean capaces de encontrar un camino indirecto de menor costo que la arista directa.

- $0 \rightarrow 1$ (peso 10)
- $0 \rightarrow 2$ (peso 1)
- $2 \rightarrow 1$ (peso 2)

4. **Grafo con ciclo de peso negativo.** Se verifica que ambos algoritmos detecten correctamente la existencia de un ciclo negativo e informen la imposibilidad de calcular distancias mínimas finitas.

- $0 \rightarrow 1$ (peso 1)
- $1 \rightarrow 2$ (peso -1)
- $2 \rightarrow 0$ (peso -1)

En todos los casos se comprobó que tanto el algoritmo base (Bellman–Ford ejecutado para cada nodo origen) como Floyd–Warshall produjeran la salida esperada o detectaran correctamente la existencia de ciclos negativos cuando correspondía.

La Figura 1 presenta los resultados obtenidos en las ejecuciones experimentales.

```

--- TEST 1: Grafo con pesos negativos (Sin ciclos) ---
Exito: los algoritmos calcularon exactamente las mismas distancias

--- TEST 2: Grafo de 5 nodos con matriz de distancias calculada a mano ---
Exito: ambos algoritmos reproducen la matriz calculada a mano (incluye pares inalcanzables)

--- TEST 3: El camino con más aristas gana a la arista directa ---
Exito: ambos algoritmos prefieren el camino indirecto más barato

--- TEST 4: Grafo CON ciclo negativo ---
Algoritmo Base detectó el problema: Se detectó un ciclo negativo accesible desde el nodo 0
Floyd-Warshall detectó el problema: El grafo contiene un ciclo de peso negativo detectado en el nodo 0

```

Figura 1: Resultados obtenidos durante las pruebas experimentales.

2.2. Análisis Experimental

Para evaluar el comportamiento práctico de los algoritmos se consideraron cuatro tipos de grafos, elegidos para cubrir casos representativos de densidad y naturaleza de los pesos:

- **Grafos densos:** grafos con una cantidad de aristas cercana al máximo posible, es decir $E = \Theta(V^2)$. Representan el peor caso en cantidad de trabajo para ambos algoritmos.
- **Grafos dispersos:** grafos con una cantidad de aristas cercana al mínimo necesario para mantener conectividad, $E = \Theta(V)$. Permiten evaluar si el rendimiento de cada algoritmo escala con V o con E .
- **Grafos en cadena:** grafos donde cada nodo se conecta únicamente con el siguiente, formando una estructura lineal sin ramificaciones. Es un caso particular de grafo disperso, pero con una estructura de dependencias mucho más rígida entre nodos.
- **Grafos negativos:** grafos que incluyen aristas con peso negativo (sin ciclos negativos), utilizados para comparar el comportamiento de los algoritmos en un escenario que Dijkstra no puede resolver correctamente.

Cada tipo de grafo se probó en instancias de 10, 20, 30, 40 y 50 nodos, realizando 32 repeticiones por instancia.

Se puede observar que, independientemente del tipo de grafo, Floyd–Warshall es más rápido que el algoritmo base. Esto se debe a su implementación basada en programación dinámica, donde se invierte en un mayor uso de espacio (la matriz de distancias completa) para evitar recomputar subproblemas y así realizar cálculos más rápidos.

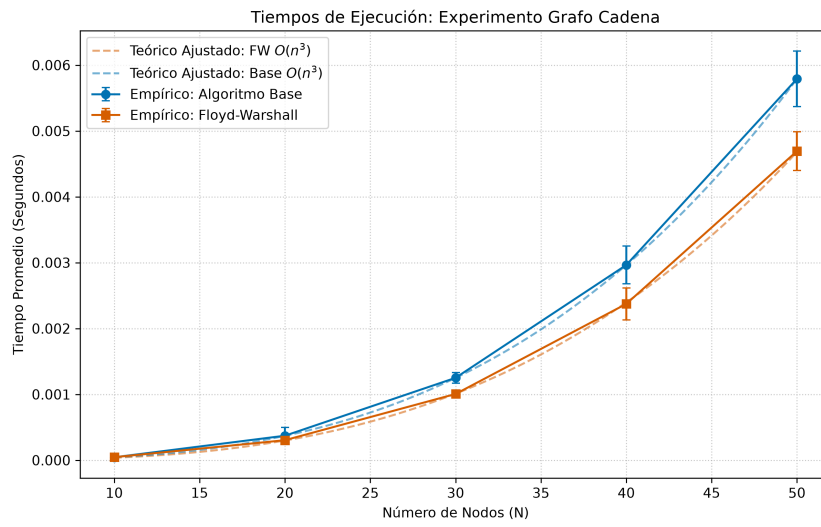


Figura 2: Resultados obtenidos para grafos cadena.

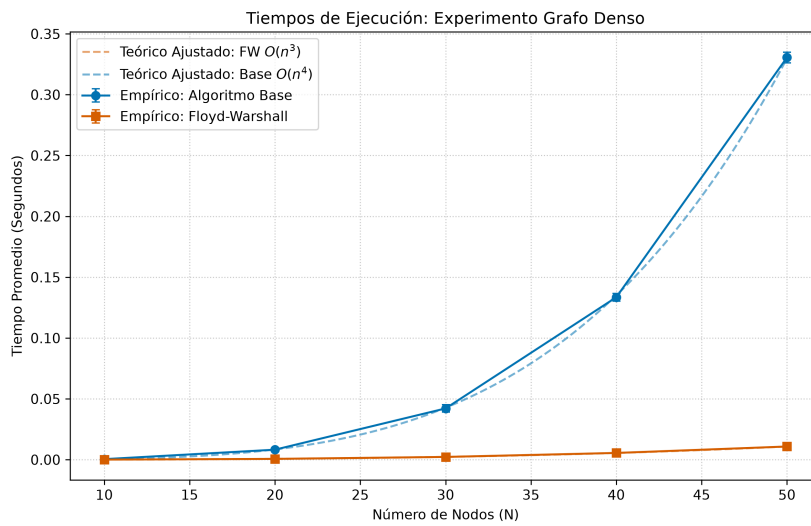


Figura 3: Resultados obtenidos para grafos densos.

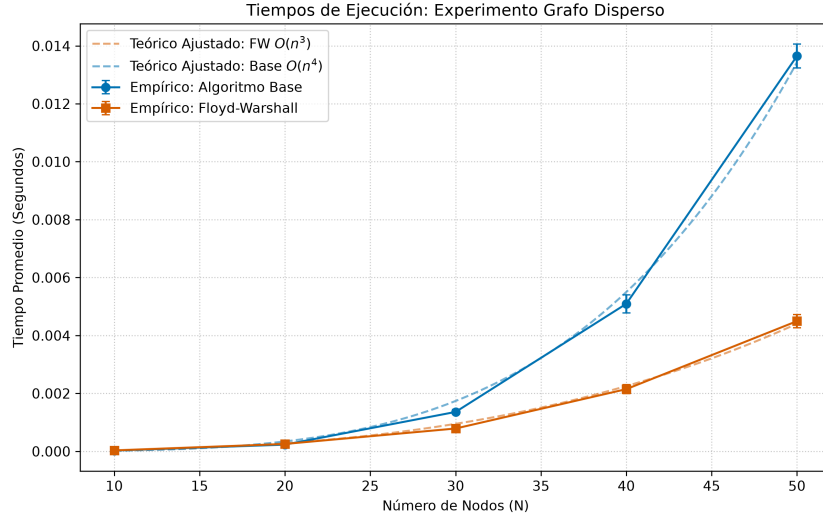


Figura 4: Resultados obtenidos para grafos dispersos.

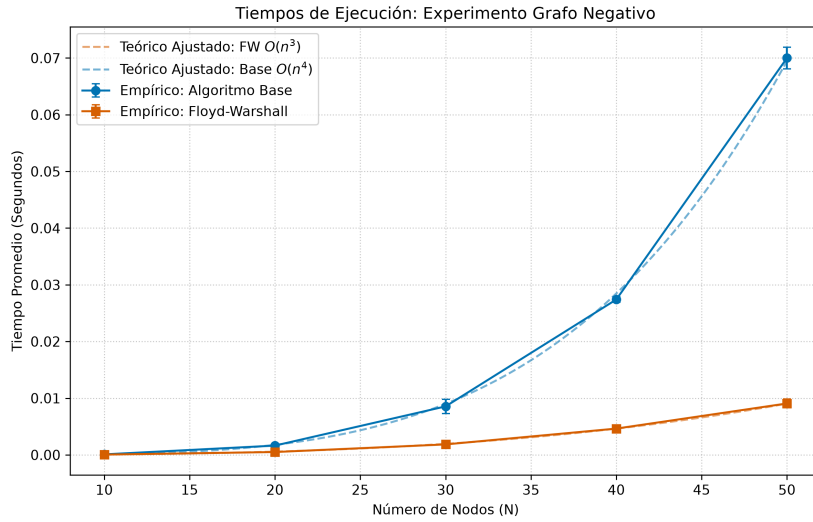


Figura 5: Resultados obtenidos para grafos negativos.

La diferencia entre ambos algoritmos es más pronunciada en los grafos densos, dispersos y negativos que en los grafos en cadena. Esto sugiere que el uso de programación dinámica resulta menos ventajoso en estructuras con pocas dependencias reutilizables entre subproblemas, como es el caso de las cadenas.

2.3. Ejecución sobre Datasets Reales

Se ejecutaron ambas alternativas propuestas en el enunciado (Chebyshev2, y el par bio-SC-TS + power-685-bus), para contar con mayor cobertura de escenarios reales.

Preprocesamiento: Cada dataset se descargó en su formato original desde Network Repository. bio-SC-TS viene como lista de aristas (nodo1 nodo2 peso) y se trató como

grafo no dirigido, dado que representa relaciones simétricas de co-expresión génica. power-685-bus y chebyshev2 vienen en formato Matrix Market (.mtx); el encabezado de cada archivo indica si la matriz es symmetric o general, lo que se usó para decidir si cada arista debía duplicarse en ambas direcciones. power-685-bus resultó no dirigido (matriz simétrica) y chebyshev2 se trató como dirigido (matriz general), descartando además las entradas de la diagonal (auto-loops).

Tratamiento de pesos: Para bio-SC-TS se usaron los pesos reales del archivo, correspondientes a puntajes de confianza (log-likelihood) en el rango [2.51, 3.54]. Para power-685-bus y chebyshev2 no se usaron los valores originales de la matriz como peso de las aristas: representan admitancias eléctricas y coeficientes de diferenciación espectral respectivamente, magnitudes sin interpretación de distancia y mayoritariamente negativas, por lo que se optó por asignar peso unitario a cada arista, conservando solo la estructura de conectividad. Se declara esta decisión explícitamente para no dar la impresión de que se descartaron datos de forma arbitraria.

Dataset	$ V $	$ E $	Pesos usados	Tiempo FW	Archivo CSV
bio-SC-TS	636	3.959 (no dir.)	Reales [2,51, 3,54]	7,14 s	bio-SC-TS.csv
power-685-bus	685	1.282 (no dir.)	Unitarios (1)	13,67 s	power-685-bus.csv
chebyshev2	2.053	16.394 (dir.)	Unitarios (1)	518,99 s	chebyshev2.csv

Cuadro 1: Resultados de ejecución de Floyd-Warshall sobre datasets reales. Los archivos se encuentran en `resultados/distancias_<dataset>.csv`.

Los resultados de cada ejecución se exportaron a un .csv independiente, con formato {nodo1, nodo2, distancia_mínima} según lo solicitado. Los pares de nodos no alcanzables se representan con el valor `inf` en la tercera columna. bio-SC-TS no es un grafo conexo (81 componentes, la mayor con 67 nodos), por lo que el 95,3% de los pares reportados en su CSV corresponden a distancia infinita; power-685-bus y chebyshev2 resultaron fuertemente conexos, sin pares inalcanzables.

3. Discusión y Extensiones

3.1. Discusión de los resultados

En todas las pruebas realizadas, los tiempos de ejecución promedio se ajustan a la complejidad asintótica teórica de cada algoritmo. Esto es consistente con que nuestras implementaciones no incorporan estructuras ni procesos adicionales (por ejemplo, operaciones de entrada/salida u operaciones auxiliares no esenciales al algoritmo) que pudieran introducir sobrecostos ajenos a la complejidad temporal analizada, por lo que el comportamiento empírico observado refleja fielmente el crecimiento asintótico esperado.

3.2. Caso especial de pesos unitarios

Si el peso de todas las aristas es igual a uno, resolver APSP no requiere acumular pesos como lo hace Floyd-Warshall, sino simplemente contar aristas: la distancia mínima entre dos nodos corresponde al camino con menor cantidad de aristas. En este caso, en lugar de Floyd-Warshall conviene ejecutar una Búsqueda en Anchura (BFS) desde cada uno de los V nodos del grafo. BFS resuelve el problema de un único origen en $O(V + E)$: al ser

todos los pesos iguales, la cola de prioridad que usa Dijkstra deja de ser necesaria, pues los nodos se exploran naturalmente en orden de distancia con una simple cola FIFO. Repetir BFS para cada nodo entrega una complejidad total de $O(V(V + E))$ para APSP, lo que representa una mejora considerable frente a los $O(V^3)$ de Floyd–Warshall, en particular en grafos dispersos ($E \ll V^2$).

3.3. Estado del arte y aplicaciones prácticas

A julio de 2026, no existe un algoritmo que resuelva APSP en tiempo verdaderamente subcúbico para el caso general de grafos con pesos reales arbitrarios (sin ciclos negativos); de hecho, en la literatura de complejidad de grano fino se conjetura que tal algoritmo no existe. El mejor resultado teórico conocido es el de Williams [1], que reduce la complejidad a $O\left(n^3/2^{\Omega(\sqrt{\log n})}\right)$, una mejora subpolinomial – y no polinomial – respecto de los $O(n^3)$ de Floyd–Warshall.

Los avances más significativos de los últimos años se han dado, en cambio, en el problema de Single-Source Shortest Path (SSSP), lo que impacta indirectamente en APSP para grafos dispersos a través del algoritmo de Johnson, que reduce APSP a n ejecuciones de SSSP. Bernstein, Nanongkai y Wulff-Nilsen [2] demostraron que SSSP con pesos negativos puede resolverse en tiempo casi lineal, lo que permite resolver APSP disperso en $\tilde{O}(nm)$. Más recientemente, Duan *et al.* [3] presentaron el primer algoritmo determinista que rompe la barrera de ordenamiento de Dijkstra para SSSP con pesos no negativos, con complejidad $O(m \log^{2/3} n)$ por fuente.

Sin embargo, estas soluciones teóricas resultan difíciles de implementar en la práctica; por esta razón, las principales librerías de grafos, como NetworkX, Boost Graph Library y JGraphT, continúan utilizando implementaciones basadas en Bellman–Ford, Floyd–Warshall o Johnson, dependiendo de las características del grafo (densidad, presencia de pesos negativos, tamaño).

4. Anexo: Códigos y Programas

Declaración de uso de herramientas de IA

Declaro que el uso de herramientas de IA generativa en este informe se ha restringido exclusivamente a tareas estéticas y de presentación (mejorar redacción, traducir términos técnicos, ajustes estéticos, corrección del informe en el formato latex).

A. Anexo: Códigos y Programas

A.1. Algoritmo Base (Bellman-Ford)

Listing 1: Implementación del Algoritmo Base

```
# src/algoritmos/base.py

def algoritmo_base(grafo):
    n = grafo.n
    aristas = grafo.obtener_aristas()
    return alg_base(n, aristas)
```



```

def alg_base(n, aristas):
    INF = float('inf') # Distancia infinita
    dist = [[INF] * n for _ in range(n)]

    for s in range(n):
        dist[s][s] = 0

        for _ in range(n - 1):
            for u, v, w in aristas:
                # Relajar la arista (u, v) con peso w
                if dist[s][u] != INF and dist[s][u] + w < dist[s][v]:
                    dist[s][v] = dist[s][u] + w

        for u, v, w in aristas:
            if dist[s][u] != INF and dist[s][u] + w < dist[s][v]:
                raise ValueError(f"Se detecto un ciclo negativo accesible desde el nodo {s}")

    return dist

```

A.2. Algoritmo de Floyd-Warshall

Listing 2: Implementación del Algoritmo de Floyd-Warshall

```

def floyd_warshall(grafo):
    # Extraemos la informacion del objeto
    n = grafo.n
    aristas = grafo.obtener_aristas()

    # Llamamos a la logica pura
    return alg_floyd_warshall(n, aristas)

def alg_floyd_warshall(n, aristas):
    INF = float('inf') # Distancia infinita
    dist = [[INF] * n for _ in range(n)]

    # Distancia de cada nodo a si mismo es 0
    for i in range(n):
        dist[i][i] = 0

    # Inicializamos las distancias directas usando la lista de aristas
    for u, v, w in aristas:
        # Si hay aristas paralelas, nos quedamos con la de menor peso
        if w < dist[u][v]:
            dist[u][v] = w

    # Probamos cada nodo k como intermediario
    for k in range(n):
        for i in range(n):
            for j in range(n):
                # Solo sumamos si no estamos en el infinito
                if dist[i][k] != INF and dist[k][j] != INF:
                    if dist[i][k] + dist[k][j] < dist[i][j]:
                        dist[i][j] = dist[i][k] + dist[k][j]

```

```

# Si dist[i][i] < 0 hay ciclo negativo
for i in range(n):
    if dist[i][i] < 0:
        # Levantamos un error en lugar de imprimir para proteger los
        # experimentos masivos
        raise ValueError(f"El grafo contiene un ciclo de peso
                           negativo detectado en el nodo {i}")

return dist

```

Referencias

- [1] R. Williams, “Faster all-pairs shortest paths via circuit complexity,” in *Proc. 46th Annu. ACM Symp. Theory of Computing (STOC)*, New York, NY, USA, 2014, pp. 664–673.
- [2] A. Bernstein, D. Nanongkai, and C. Wulff-Nilsen, “Negative-weight single-source shortest paths in near-linear time,” in *Proc. 63rd IEEE Annu. Symp. Foundations of Computer Science (FOCS)*, 2022, pp. 600–611.
- [3] R. Duan, J. Mao, X. Mao, X. Shu, and L. Yin, “Breaking the sorting barrier for directed single-source shortest paths,” in *Proc. 57th Annu. ACM Symp. Theory of Computing (STOC)*, Prague, Czechia, 2025, pp. 36–44.